

EduCPU-8 — 10 Challenges & Solutions

Companion answer key for the "10 Challenges for Students" slide in [EduCPU8-Teacher-Presenter.pptx](#). Each challenge builds on a worked example already in the tool — same nine instructions ([LOAD](#), [STORE](#), [ADD](#), [SUB](#), [JMP](#), [JZ](#), [OUT](#), [OUTC](#), [HALT](#)), new problem.

How to use this doc: paste a solution's code into the EduCPU-8 editor, set the memory cells listed under "Starting memory," click **Assemble & Load**, then **Step** or **Run**. Every solution below has been verified against the actual assembler and CPU, so the listed output is exactly what you'll see.

These are *sample* solutions, not the only correct ones — if a student's program produces the right output a different way, that's a win, not a problem.

1. Print your initials

Level: Warm-up · **Skills:** [LOAD](#) [#'...'](#), [OUTC](#)

Print your own initials to the console. No memory or loop needed — this is Output on its own, like the Hello Text example.

Starting memory: none

```
; Print your initials, one character at a time.  
LOAD #'A'  
OUTC  
LOAD #'B'  
OUTC  
HALT
```

Output: [AB](#)

2. Triple a number

Level: Warm-up · **Skills:** [LOAD](#), [ADD](#), [STORE](#), [OUT](#)

Multiply the value in memory cell 0 by three, store the result in memory cell 1, and print it. There's no multiply instruction yet — just add the value to itself.

Starting memory: [0](#) → [5](#)

```
; Triple the value in memory 0.  
LOAD 0  
ADD 0  
ADD 0  
STORE 1
```

```
OUT
HALT
```

Output: 15 (5 × 3)

3. Is it zero?

Level: Warm-up · **Skills:** LOAD, JZ, branching

Print YES if memory cell 0 is exactly 0, otherwise print NO. This is the simplest possible branch — one JZ, no SUB needed.

Starting memory: 0 → 0

```
; Print YES if memory 0 is zero, else NO.
LOAD 0
JZ 9
LOAD #'N'
OUTC
LOAD #'O'
OUTC
JMP 15
LOAD #'Y'
OUTC
LOAD #'E'
OUTC
LOAD #'S'
OUTC
HALT
```

Output: YES when memory 0 is 0; change it to any non-zero value (e.g. 5) and re-run to see NO instead.

4. Count up to a target

Level: Core · **Skills:** loop, ADD #1, JZ to stop

Count up from memory cell 0 to memory cell 1 (inclusive), printing every number along the way. The mirror image of the Skip Counting example.

Starting memory: 0 → 0, 1 → 3

```
; Count up from memory 0 to memory 1 (inclusive).
LOAD 0
OUT
SUB 1
JZ 10
LOAD 0
```

```
ADD #1
STORE 0
JMP 2
HALT
```

Output: 0 1 2 3

5. Sum from 1 to N

Level: Core · **Skills:** loop + a running total in memory

Add up $1 + 2 + \dots + N$, where N is stored in memory cell 0. Memory cell 1 holds the running total as the loop goes — the same "counter plus accumulator" shape as Multiply by Repeated Addition, applied to a different problem.

Starting memory: 0 → 4

```
; Sum 1 + 2 + ... + N, where N is in memory 0. Running total in memory 1.
LOAD 0
JZ 11
LOAD 1
ADD 0
STORE 1
LOAD 0
SUB #1
STORE 0
JMP 2
LOAD 1
OUT
HALT
```

Output: 10 (1 + 2 + 3 + 4)

6. Halve a number by counting

Level: Core · **Skills:** repeated SUB, counting how many times

There's no divide instruction either — so this finds $\text{memory } 0 \div 2$ by repeatedly subtracting 2 and counting how many subtractions it took. Memory cell 1 holds that count, which is the answer.

Starting memory: 0 → 8

```
; Halve memory 0 by repeatedly subtracting 2 and counting how many times.
LOAD 0
JZ 10
SUB #2
STORE 0
```

```
LOAD 1
ADD #1
STORE 1
JMP 2
LOAD 1
OUT
HALT
```

Output: 4 (8 ÷ 2)

Discussion point: this only lands on exactly 0 if memory 0 is even. Try it with an odd starting value (e.g. 7) and watch it run forever without halting — a real example of why loop conditions matter, and a good moment to click **Pause**.

7. Are three numbers all equal?

Level: Core · **Skills:** two chained comparisons

Print **YES** only if memory cells 0, 1 and 2 all hold the same value, otherwise **NO**. Extends Are They Equal? from one comparison to two, chained: it only reaches the second check if the first one passes.

Starting memory: 0 → 5, 1 → 5, 2 → 5

```
; Print YES only if memory 0, 1 and 2 are all equal.
LOAD 0
SUB 1
JZ 6
JMP 10
LOAD 1
SUB 2
JZ 15
JMP 10
LOAD #'N'
OUTC
LOAD #'O'
OUTC
JMP 21
LOAD #'Y'
OUTC
LOAD #'E'
OUTC
LOAD #'S'
OUTC
HALT
```

Output: **YES** for 5, 5, 5. Change memory cell 2 to 3 (first pair still equal, second pair not) or memory cell 1 to 3 (first pair already unequal) — both print **NO**, but take different paths through the code.

8. Countdown, then shout GO!

Level: Core · **Skills:** a loop followed by fixed text output

Count down from memory cell 0 to 1 like the Countdown Loop example, but once the countdown finishes, print **GO!** — combining a loop with an unrolled character sequence, the two output techniques from earlier in the deck, in one program.

Starting memory: 0 → 3

```
; Count down from memory 0 to 1, then print GO!
LOAD 0
JZ 8
OUT
SUB #1
STORE 0
JMP 2
LOAD #'G'
OUTC
LOAD #'O'
OUTC
LOAD #'!'
OUTC
HALT
```

Output: 3 2 1 GO!

9. Predict the overflow

Level: Stretch · **Skills:** two's complement, **ADD**

Before running this one, predict on paper what each of the three **OUT** lines will print. Then load it and Step through to check.

Starting memory: 0 → 90

```
; Start at 90, add 20 twice. Predict both printed values before you Step.
LOAD 0
OUT
ADD #20
OUT
ADD #20
OUT
HALT
```

Output: 90 110 -126

Why: EduCPU-8's ACC is a genuine signed byte, clamped to $-128..127$. $90 + 20 = 110$ fits fine. But $110 + 20 = 130$ doesn't — it's past the top of the range, so it wraps around to -126 (the same way $127 + 1$ becomes -128 in the Overflow! example). Watch ACC's bit row in the tool: the pattern doesn't reset to all zeros, it just gets reinterpreted as negative.

10. Design your own

Level: Stretch · **Skills:** combine instructions nobody told you to combine

Open-ended — pick at least two instructions you haven't used together yet and build something new. There's no single correct answer; below is just one sample so you can see the shape a "your own" answer might take.

Sample answer — computes $(\text{memory } 0 + \text{memory } 1) - \text{memory } 2$, stores the result in memory cell 3, and prints it:

Starting memory: $0 \rightarrow 10, 1 \rightarrow 7, 2 \rightarrow 3$

```
; Sample answer – yours will look different! Computes (mem0+mem1)-mem2.
LOAD 0
ADD 1
SUB 2
STORE 3
OUT
HALT
```

Output: 14 $(10 + 7 - 3)$